

SISTEMA DE CRAFTEO PROCEDURAL

Especificación funcional y técnica para integración en el game engine

Objetivo: permitir que el jugador fabrique armas únicas mediante un sistema procedural controlado, donde el jugador decide qué desea fabricar pero no el resultado estadístico exacto. El sistema debe producir variación sin convertir el crafting en una lotería de poder.

1. Principio general del sistema

El crafeo se basa en una separación estricta de responsabilidades. Cada variable debe responder a una sola pregunta de diseño. Esto evita que Tier, calidad, rareza y material terminen representando la misma idea de «qué tan buena es el arma».

Variable	Responsabilidad
Tier	Cuánto poder puede contener el arma. Lo define la habilidad de crafting del personaje.
Tipo de arma	Cómo se distribuye naturalmente ese poder: daño, velocidad, crítico, penetración, alcance, etc.
Material	Cómo se modifica el comportamiento del arma. No representa calidad.
Generación procedural	Redistribuye una pequeña parte del presupuesto para crear armas diferentes dentro del mismo Tier.
Añijo / componente	Añade una propiedad especial que normalmente no pertenece a las estadísticas base.
Nombre	Lo introduce el jugador después de crear el arma.

2. Tier: capacidad del artesano

El Tier no es una rareza aleatoria. Representa directamente el nivel técnico que el personaje puede fabricar. Si el personaje domina Tier III, una fabricación Tier III ya debe considerarse una ejecución competente de ese nivel.

Por esta razón, no se recomienda una variable adicional de calidad del tipo Defectuosa / Común / Buena / Maestra si esa calidad también depende de la habilidad de crafting. Sería una duplicación conceptual.

Nivel de crafting	Tier máximo	Interpretación
1	I	Armas básicas y técnicas simples
2	II	Construcción competente
3	III	Armas avanzadas
4	IV	Fabricación excepcional
5	V	Maestría artesanal

3. Presupuesto de poder

Cada Tier recibe un presupuesto de poder abstracto. Este valor no tiene que mostrarse al jugador. Su función es permitir al generador mantener todas las armas de un mismo Tier dentro de una banda de potencia comparable.

Ejemplo inicial de tuning: Tier I = 100, Tier II = 130, Tier III = 170, Tier IV = 220, Tier V = 290. Estos valores son parámetros de balance y pueden cambiar sin alterar la arquitectura.

Regla de seguridad: la generación aleatoria no debe modificar libremente el poder total. Debe redistribuirlo entre estadísticas. Como referencia, una desviación máxima de $\pm 10\%$ a $\pm 15\%$ por estadística es suficiente para crear identidad sin generar power creep.

4. Tipo de arma

El jugador elige el tipo de arma. El tipo define un perfil base de distribución del presupuesto y las estadísticas disponibles.

Tipo	Tendencia base	Ejemplo de identidad
Espada	Equilibrada	Daño medio, velocidad media, crítico medio, penetración media
Maza	Impacto / armadura	Daño alto, lenta, buena penetración o daño contra armadura
Daga	Velocidad / crítico	Daño bajo por golpe, alta velocidad, crítico alto
Lanza	Alcance / penetración	Daño medio, alcance superior, buena penetración

5. Material

El material no define «qué tan buena» es el arma. Define **cómo se comporta**. Una espada Tier V de hierro sigue siendo Tier V: simplemente expresa su poder de forma distinta a una espada Tier V de acero.

Material	Ejemplo de efecto
Hierro	+ daño físico o masa; + peso; posible penalización de velocidad
Acero	+ penetración y durabilidad; comportamiento equilibrado
Bronce	+ facilidad / velocidad; - penetración o durabilidad máxima
Plata	+ eficacia contra criaturas sobrenaturales; menor resistencia física
Obsidiana	+ crítico y/o sangrado; durabilidad baja

6. Generación procedural

La generación procedural crea diferencias entre dos armas que comparten Tier, tipo y material, sin establecer una jerarquía automática de «arma buena» y «arma mala».

Ejemplo: dos lanzas Tier III de acero pueden tener cantidades equivalentes de poder, pero una puede sacrificar algo de daño para ganar penetración, mientras otra gana velocidad y pierde alcance efectivo.

	Lanza A	Lanza B
Daño	32	28
Velocidad	1.00	1.08
Crítico	5 %	7 %
Penetración	14	11
Resultado	Equilibrada	Más rápida / precisa

Perfil interno

El engine puede asignar un perfil interno como «equilibrado», «pesado», «rápido», «preciso» o «perforante». Este perfil sirve como plantilla de redistribución, pero no necesita mostrarse al jugador ni actuar como una rareza.

Ejemplo: perfil Pesado = +10 % daño, -7 % velocidad, -3 % crítico. Perfil Perforante = -5 % daño, +12 % penetración, -3 % velocidad. El sistema debe conservar aproximadamente el mismo presupuesto total.

7. Afijos como componentes especiales

Los afijos deben provenir de componentes opcionales obtenidos principalmente durante la exploración o el modo dungeon. Su función es introducir propiedades únicas, no inflar de forma genérica todas las estadísticas.

Componente	Propiedad posible
Glándula venenosa	Probabilidad de aplicar Veneno
Colmillo de bestia	Sangrado o bonificación contra enemigos heridos
Corazón de salamandra	Daño adicional de fuego
Fragmento rúnico	Daño o penetración mágica
Núcleo cristalino	Efecto elemental, energía o resistencia especial

Regla recomendada: un afijo puede consumir una parte del presupuesto total del arma o utilizar un presupuesto separado muy controlado. No conviene que sea poder gratuito ilimitado, porque haría que toda arma sin afijo fuese automáticamente inferior.

8. Flujo de crafting

1. El jugador obtiene materiales y componentes en Dungeon.
2. El jugador abre la interfaz de crafting.
3. El sistema obtiene el Tier máximo desde la habilidad de crafting.
4. El jugador elige tipo de arma.
5. El jugador elige material.
6. El jugador puede añadir un componente especial opcional.
7. El engine calcula el presupuesto de poder del Tier.
8. La plantilla del tipo distribuye el presupuesto.
9. El material aplica modificadores.
10. La generación procedural redistribuye pequeñas cantidades de poder.

11. El componente añade el afijo correspondiente.
12. Se validan límites de balance.
13. Se crea la instancia del arma.
14. El jugador introduce el nombre.

9. Estructura de datos sugerida

```
weapon_instance = {
    "id": unique_id,
    "name": player_name,
    "tier": 3,
    "weapon_type": "spear",
    "material": "steel",
    "internal_profile": "piercing",
    "stats": {
        "damage_min": 28,
        "damage_max": 36,
        "speed": 0.94,
        "crit": 0.05,
        "penetration": 18,
        "range": 2
    },
    "affix": {
        "id": "venom",
        "proc_chance": 0.05
    }
}
```

10. Pseudocódigo de generación

```
def crear_arma(personaje, tipo, material, componente=None):

    tier = personaje.crafting_tier
    presupuesto = TIER_POWER[tier]

    plantilla = WEAPON_TYPES[tipo]
    stats = distribuir_presupuesto(plantilla, presupuesto)

    stats = aplicar_material(stats, material)

    perfil = elegir_perfil_valido(tipo)
    stats = redistribuir_stats(stats, perfil, limite=0.10)

    affix = None
    if componente is not None:
        affix = generar_afijo(componente, tier)
        stats = compensar_coste_afijo(stats, affix)

    stats = validar_limites(stats, tipo, tier)

    arma = Weapon(
        tier=tier,
        weapon_type=tipo,
        material=material,
        internal_profile=perfil,
        stats=stats,
        affix=affix,
        name=None
    )

    return arma
```

11. Ejemplos de salida

Espada Tier III - Acero

Daño 31-43 | Velocidad 0.96 | Crítico 5 % | Penetración 15

Perfil interno: perforante. Sin afijo.

Maza Tier III - Hierro

Daño 38-52 | Velocidad 0.78 | Crítico 3 % | Penetración 19

Perfil interno: pesada. Sin afijo.

Daga Tier III - Obsidiana

Daño 18-27 | Velocidad 1.38 | Crítico 12 % | Penetración 7

Perfil interno: precisa. Afijo opcional: sangrado.

Lanza Tier III - Acero

Daño 27-36 | Velocidad 0.91 | Crítico 5 % | Penetración 18 | Alcance 2

Perfil interno: perforante. Afijo opcional: veneno.

12. Integración con Dungeon y combate táctico

La instancia del arma debe ser única y compartida por ambos modos. No se recomienda guardar una versión «Dungeon» y otra «Táctica» del arma. Ambos sistemas deben leer los mismos datos base y aplicar sus propias fórmulas de interpretación.

Ejemplo:

Arma base: daño 20-40, velocidad 1.0, penetración 12.

Dungeon: puede utilizar directamente el rango 20-40 y resolver golpes individuales.

Combate táctico: puede utilizar el daño medio 30 como base y después aplicar cobertura, flanco, armadura, habilidades o posición.

La progresión del arma sigue siendo la misma aunque el resultado numérico de cada modo se calcule de forma distinta.

13. Reglas de balance obligatorias

- Un Tier superior debe aumentar la capacidad total del arma, no necesariamente todas sus estadísticas.
- Dos armas del mismo Tier, tipo y material deben poder ser diferentes sin que una sea siempre objetivamente superior.
- La aleatoriedad debe redistribuir poder, no crear grandes cantidades de poder nuevo.
- El material debe cambiar el comportamiento, no reemplazar al Tier.
- Los afijos deben tener coste, límite o presupuesto independiente controlado.
- El nombre del arma nunca afecta estadísticas.
- Los mismos datos del arma deben alimentar Dungeon y combate táctico.
- Las tablas de Tier, tipos, materiales, perfiles y afijos deben ser data-driven para permitir balance sin cambiar código.

14. Arquitectura recomendada

Separar los datos estáticos de las instancias generadas. WEAPON_TYPES, MATERIALS, TIER_POWER, INTERNAL_PROFILES y AFFIXES deben vivir en recursos/JSON/configuración. El generador solo combina esas reglas y produce un WeaponInstance serializable.

Resultado buscado: el jugador controla la intención de fabricación; el sistema controla la variación. Tier responde a habilidad, tipo responde a función, material responde a comportamiento, generación responde a identidad, afijo responde a propiedades especiales y el jugador aporta el nombre.